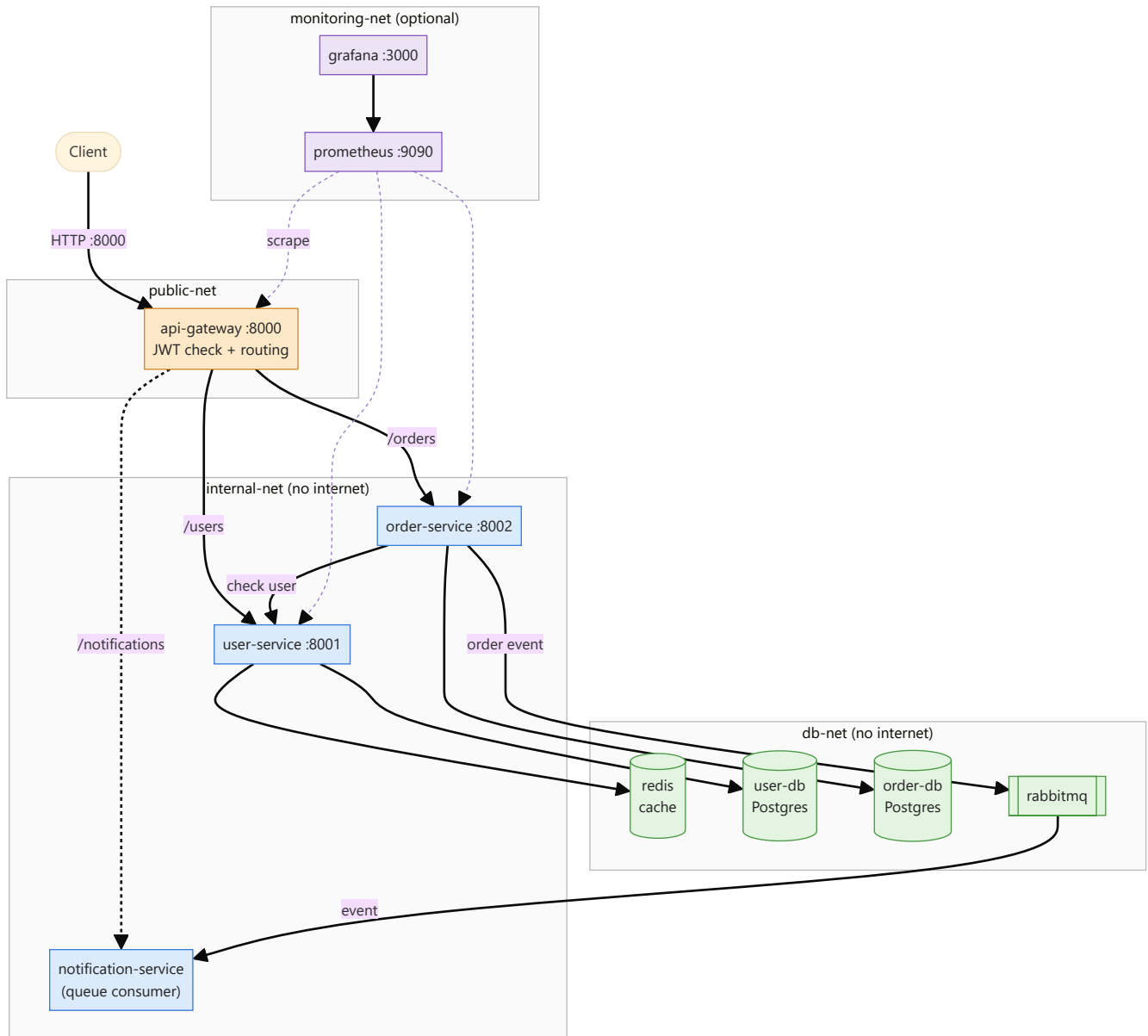


Deployment Architecture

go-microservices · based on docker-compose.yml, docker-compose.monitoring.yml and .github/workflows/ci-cd.yml · 15 Sep 2026



Everything above runs on one Docker host. Purple dotted = Prometheus reading metrics. Black dotted = route exists but does not work (see notes).

How it works, in plain terms

- **One entry point.** All outside traffic goes to the **api-gateway** on port 8000. It checks the login token (JWT) and forwards the request to the right service.
- **Three Go services** do the actual work: **user-service** (accounts and login), **order-service** (orders) and **notification-service** (sends order notifications).
- **Each service owns its data.** user-service uses its own Postgres database plus Redis to cache user lookups for 10 minutes. order-service uses a separate Postgres database.
- **Orders trigger notifications through a queue.** When an order is created, order-service drops a message into RabbitMQ. notification-service picks it up. The order request does not wait for the

notification.

- **order-service asks user-service** directly over HTTP to check user details when creating an order.

Networks (who can talk to whom)

Network	Members	Purpose
public-net	api-gateway	The only app container reachable from outside (host port 8000).
internal-net	gateway, 3 services, prometheus	Service-to-service calls. Cut off from the internet.
db-net	3 services, 2 Postgres, Redis, RabbitMQ	Data stores. Not reachable by the gateway or from outside.
mgmt-net	rabbitmq	Only exists so the RabbitMQ admin page can be opened on localhost:15672.
monitoring-net	prometheus, grafana	Dashboards. Grafana cannot reach app services directly.

Ports opened on the host

Port	Container	Who should use it
8000	api-gateway	Clients
127.0.0.1:15672	rabbitmq admin UI	Developer, local machine only
5672	rabbitmq (AMQP)	Open on all interfaces — see notes
9090 / 3000	prometheus / grafana	Developer (only when the monitoring file is used)

How it is started

- **App only:** `docker compose up -d --build`
- **App + monitoring:** `docker compose -f docker-compose.yml -f docker-compose.monitoring.yml up -d --build`
- Data is kept in named Docker volumes (`user_db_data`, `order_db_data`, `redis_data`, `rabbitmq_data`, `grafana-data`), so it survives container restarts.
- Services start in order: databases and RabbitMQ must pass their health checks before the Go services start.

CI pipeline (GitHub Actions)

On every push to `main`: build and start everything with Docker Compose on a GitHub runner → run the Node test suite against `localhost:8000` → check that `/metrics` responds → print logs if anything failed → shut down and delete volumes. **Nothing is deployed to a server**; the pipeline only builds and tests. `terraform/terraform.yml` is empty, so there is no cloud setup yet.

Things worth knowing

- Everything runs on **one machine**. There is one copy of each service, so if a container or the host goes down, that part is unavailable until it restarts.
- RabbitMQ port **5672 is published on all interfaces**, unlike the admin UI which is bound to `localhost`.
- The gateway routes `/notifications` to `notification-service:8003`, but `notification-service` does not run an HTTP server — it only reads from the queue. Those requests will fail (dotted line in the diagram).
- Prometheus does not scrape `notification-service`.
- Passwords and secrets (Postgres, JWT, Grafana admin, RabbitMQ `guest`) are hard-coded in the compose files and Go code.